✅ T0 - ML/MLOps Engineering Internship

Task 0 Technical Assessment (60 minutes)

Goal

Build a **minimal MLOps-style batch job** in Python that demonstrates:

- **Reproducibility** (deterministic runs via config + seed)
- **Observability** (logs + machine-readable metrics)
- **Deployment readiness** (Dockerized, one-command run)

This mirrors the type of work we do in **MetaStackerBandit** (trading-signal pipelines).

What you'll build

A small Python program that:

1. Loads config from YAML
2. Reads a provided `data.csv` (10,000 rows OHLCV)
3. Computes a rolling mean on **close**
4. Generates a binary signal from close vs rolling mean
5. Writes structured metrics JSON + detailed logs
6. Runs locally and inside Docker

Required CLI

Your program must run as:

python run.py --input data.csv --config config.yaml --output metrics.json --log-file run.log

No hard-coded paths.

Input files

Config: `config.yaml`

Must include:

seed: 42
window: 5
version: "v1"

Dataset: `data.csv`

Contains OHLCV columns. **Use only `close` for calculations.**
Validate that `close` exists and input is readable/non-empty.

Processing requirements (run.py)

Implement these steps:

1) Load + validate config

- Parse YAML, validate required fields (`seed`, `window`, `version`)
- Set seed: `numpy.random.seed(seed)` (or equivalent)

2) Load + validate dataset

Handle these cases cleanly:

- Missing input file
- Invalid CSV format
- Empty file
- Missing required column (`close`)
- Invalid config structure

3) Rolling mean

Compute rolling mean on `close` using `window` from config.

Important: define how you handle the first `window-1` rows (e.g., allow NaNs and exclude from signal computation, or fill—just be consistent).

4) Signal

For each row:

- `signal = 1` if `close > rolling_mean`
- else `signal = 0`

5) Metrics + timing

Compute:

- `rows_processed`
- `signal_rate` = mean(signal)
- `latency_ms` = total runtime in milliseconds

Output: metrics.json

Success output (exact keys)

```
{
  "version": "v1",
  "rows_processed": 10000,
  "metric": "signal_rate",
  "value": 0.4990,
  "latency_ms": 127,
  "seed": 42,
  "status": "success"
}
```

Error output

```
{
  "version": "v1",
  "status": "error",
  "error_message": "Description of what went wrong"
}
```

✅ Metrics file must be written in both success and error cases.

Logging (run.log)

Use Python logging. Must include:

- Job start timestamp
- Config loaded + validated (seed/window/version)
- Rows loaded
- Processing steps (rolling mean, signal generation)
- Metrics summary
- Job end + status
- Any exceptions / validation errors

Docker requirement (must pass)

We will evaluate by running exactly:

docker build -t mlops-task .
docker run --rm mlops-task

Docker container must:

- Include `data.csv` and `config.yaml`
- Produce `metrics.json` and `run.log`
- Print final metrics JSON to stdout
- Exit code: `0` success, non-zero failure

Suggested base image: `python:3.9-slim`

Deliverables (repo)

Must include:

- `run.py`
- `config.yaml`
- data.csv (provided)
- `requirements.txt`
- `Dockerfile`
- `README.md`
- `metrics.json` (sample output from successful run)
- `run.log` (sample log from successful run)

README must include

- Local run instructions
- Docker build/run commands
- Example `metrics.json`

Evaluation rubric

- **Correctness & determinism (40%)**: signal logic, correct metrics format, reproducible results
- **Dockerization (25%)**: builds + runs cleanly, no hardcoded paths
- **Code quality (20%)**: clean structure, validation, error handling
- **Observability (15%)**: meaningful logs + error reporting

Auto-fail

- Docker build/run fails
- Metrics JSON not written
- Non-deterministic outputs
- Hardcoded paths or missing README steps

How to Apply

Email your **resume + submission link *(GitHub)*** + **log files** to:

- joydip@anything.ai
- chetan@anything.ai
- hello@anything.ai
- CC: sonika@anything.ai